Beyond the Prompt

Teaching Your AI to Think Like Your Team
and Level Up with Skills

Eric Winter · Research Genomics · 2026-04-17

Agenda

- The limits of prompt engineering
- Spec-Driven Development (recap)
- Constitutions – teaching your AI your standards
- Memory – context that learns and grows
- Rules – breaking the constitution into focused pieces
- Skills – reusable AI capabilities you can build and share
- Commands – automating workflows, not just guiding them
- Bringing it all together

How Most Developers Use AI Today

The familiar pattern

- "How do I write a regex for emails?"
- "Explain this error message"
- Inline autocomplete for the current line
- Tab to accept, keep typing

What it feels like

- A very fast Google with context
- A rubber duck that writes code
- A pair programmer who stays in their lane

> The model helps — but *you* are still writing the software.

The Paradigm Shift: Agents That *Do* the Work

Old model (assistant)

- You write code, AI autocompletes
- You ask questions, AI answers
- You drive, AI is a GPS

New model (agent)

- You describe intent, AI implements
- Agent reads your codebase, runs tests, iterates
- You review, AI does the labor

Claude Code, Copilot Workspace, Devin, Cursor Composer

These agents open files, run terminal commands, edit code across the whole repo, and loop until the tests pass — without you touching a keyboard.

But who’s keeping it on track?

Iterative refinement

- Review the diff, redirect the agent
- "That’s wrong – do it this way instead"
- Works, but requires you to catch every mistake

Context guardrails ← *this talk*

- Teach the agent your standards upfront
- It applies them without being asked
- Scales across your team, not just you
- Speeds up iterations

> Iterative refinement keeps *one task* on track. Context guardrails keep *every task* on track.

The Limits of Prompt Engineering

The Problem with Simple Prompts

- Large-context tools (Claude Code, Copilot) give the agent your entire codebase
- More context ≠ better results — without focus, the model can’t prioritize
- "Add a login page" leaves scope, constraints, and standards entirely up to the model
- Result: hallucinated APIs, ignored conventions, features that drift from intent

Hallucination Gets Worse at Scale

- Larger context = more surface area for the model to go off-track
- Without anchors, the model guesses – confidently and plausibly
- Mistakes compound: one wrong assumption early leads to a cascade

The Fix: Structured Focus

Three complementary layers that narrow the solution space:

- **Spec** – defines *what* to build and the acceptance criteria
- **Constitution** (AGENTS.md / CLAUDE.md) – defines *how* to build it
- **Skills** – enumerate *specific capabilities* the agent should apply

Together these shift the model's attention from guessing your intent to executing it.

Spec-Driven Development

Ground the AI in Specs Before Writing Code

- Maintain living documents: functional specs, acceptance criteria, prioritized backlog
- AI reads these as context — knows *what* to build, *why*, and what's out of scope
- Eliminates scope creep and hallucinated features
- Spec is consumed and completed; constitutions and skills persist indefinitely

Backlog.md – Specs in Your AI Tool

- Tasks created from specs before implementation begins
- Agent queries the backlog at session start for context and constraints
- Agent marks tasks done and updates spec docs as work completes
- Backlog lives in the repo – whole team shares the same picture

Constitutions

Teaching Your AI Your Standards

What a Constitution Is

- A structured Markdown file read by the agent at the start of every session
- The single source of truth for how work gets done on this project
- Not a prompt — a persistent, version-controlled contract between the team and the AI
- If it isn't accurate enough for a new engineer to follow, it isn't accurate enough for the agent

What Goes In a Constitution

- **Project structure** — directory layout, what belongs where, non-obvious conventions
- **Stack** — canonical libraries for every concern; versions where they matter; off-limits libs
- **Standards** — style guides, linting rules, naming conventions, architectural rules
- **Workflow** — how PRs work, how tests run, what done looks like

Personal → Team → Organizational

- Start personal: encode your own preferences and workflow
- Extend to the team: shared standards, stack decisions, naming conventions
- Grow to the org: cross-team invariants, security requirements, compliance rules
- Constitutions must be living documents — stale constitutions mislead

Demo: Constitution in Action

```
# Explore the TeamFabric root constitution
cat TeamFabric/CLAUDE.md

# Explore the template team constitution
cat TeamFabric/Fabric/template/CLAUDE.md

# Explore the core behavioral rules
cat TeamFabric/Fabric/template/fabric-core.md
```

Prompt: *"Add a new team member named Alex Chen who joins the platform engineering team as a senior engineer at 100% allocation."*

TeamFabric: Layered Constitution

Framework rules are imported. Team rules live below the line — updates never overwrite them.

```
@.claude/fabric-core.md
@.claude/fabric-triage.md
@.claude/fabric-product.md
@.claude/fabric-standup.md
@.claude/fabric-retro.md

# {{TEAM_NAME}}

<!--
  Everything below this line is yours.
  TeamFabric updates will not touch this section.

  TO CUSTOMIZE BEHAVIOR:
    - Override or extend rules here, below the @imports
    - Add team-specific commands to .claude/commands/
    - Add team-specific skills to .claude/skills/
-->
```

TeamFabric: Core Rules

One focused file. One concern: how entities work, what meta mode is, who can edit what.

```
## Meta Mode

Structural files are read-only during normal operations.
Edits require meta mode.

Entering meta mode:
- User explicitly invokes `/meta`
- AI may suggest it when a structural change is implied

## Entity File Structure

1. **Lightweight header** — cheap to load, identifies relevance quickly
2. **First-class structured fields** — curated, human-guided artifacts
3. **Context log** — append-only breadcrumb trail

### Context Log Entry Format
- YYYY-MM-DD HH:MM - Who (contact) via channel: Summary.
  Source: [reference to original artifact]
```

Memory

Context That Learns and Grows

What Memory Is

- Structured Markdown files the agent writes and reads across sessions
- Captures what the agent has learned: preferences, decisions, feedback, external pointers
- Lives in `.claude/memory/` – loaded selectively, not dumped wholesale
- Unlike a constitution, you don't author memory directly – the agent maintains it

> "You seem to be correcting me a lot on that I am going to remember this."

The Four Types of Memory

Type	What it captures	Example
`user`	Who you are, how you work	"Senior Go engineer, new to React"
`feedback`	Corrections that should stick	"Never mock the database in tests"
`project`	Decisions, motivations, not in git	"Auth rewrite is compliance-driven"
`reference`	Pointers to external systems	"Bugs in Linear project INGEST"

Constitution vs. Memory

	Constitution	Memory
Authored by	Human team	Agent
Reviewed via	PRs	Your oversight
Answers	How do we always work?	What has this agent learned?
Signal you need it	Re-explaining setup every session	Correcting the same thing across sessions

TeamFabric: Breadcrumb Memory

No raw content retained. Every interaction leaves a structured, sourced breadcrumb.

```
## Entity File Structure

Each entity uses a layered information architecture:

1. **Lightweight header** — cheap to load, identifies relevance
2. **First-class fields** — curated artifacts maintained through refinement
3. **Context log** — append-only breadcrumb trail of sourced summaries

### Context Log Entry Format

- YYYY-MM-DD HH:MM - Who (contact) via channel: Summary with reasoning.
  Source: [reference to original artifact]
```

Rules

Breaking the Constitution Into Focused Pieces

What a Rule Is

- A focused Markdown file that encodes one specific concern
- Lives in `.claude/rules/` — discovered and loaded automatically
- Each rule has a **scope** that controls when it is loaded
- The constitution becomes an index; rules become the chapters

A monolithic CLAUDE.md loads everything, every time — most of it irrelevant.
Rules keep the active context lean.

The Four Rule Scopes

Scope	When it loads	Best for
Always	Every session	Core conventions, non-negotiables
Auto-attached	File matching glob is in context	Language-specific, framework patterns
Agent-requested	Agent reads description and pulls	Deep reference, optional style guides
Manual	User explicitly references it	One-off tasks, migration guides

How the Agent Decides What to Load

The agent never sees full rule/skill content upfront — it reads **descriptions first**, then pulls the full file only if relevant.

What loads eagerly (cheap)

- Rule/skill name
- One-line `description` field
- Glob pattern (for auto-attached rules)

What loads on demand (expensive)

- Full rule body
- Full skill instructions
- Examples and constraints

The description is the decision gate.
Write it as a trigger condition, not a label.

Instead of…	Write…
`"Python style guide"`	`"Use when editing any .py file or reviewing Python PRs"`

Rules vs. Skills

	Rules	Skills
Role	Extend and modularize the constitution	Define capabilities invoked on demand
When active	Always in the background	Called in when needed
Best for	Standards and constraints	Specific task playbooks

Rules are your team's standing orders.
Skills are the specialist playbooks you hand out for specific missions.

TeamFabric: Module System as Rules

Each module is a scoped rule file. Teams opt in at init time — only relevant rules load.

```
@.claude/fabric-core.md       ← always loaded (core rules)
@.claude/fabric-triage.md     ← loaded when Triage enabled
@.claude/fabric-product.md    ← loaded when Product enabled
@.claude/fabric-standup.md    ← loaded when Standup enabled
@.claude/fabric-retro.md      ← loaded when Retro enabled

## Enabled Modules

| Module   | Status   | Notes                                          |
|──────────|──────────|────────────────────────────────────────────────|
| Core     | Enabled  | Team definition, members, ingestion            |
| Triage   | Enabled  | Request intake, rubric evaluation              |
| Product  | Enabled  | Product definitions and context                |
| Backlog  | Disabled | Epic/feature/work-item hierarchy               |
| Standup  | Disabled | Daily standup conversations                    |
```

TeamFabric: A Rule File

One file. One module. Everything the agent needs to know about request triage — nothing else.

```
# TeamFabric Module: Triage

## Overview
The Triage module manages request intake, evaluation workflows,
and rubric-based assessment. Teams customize the specific rubrics
and workflow definitions in `requests/workflow/`.

## Behavioral Rules

- New requests are created in `requests/<request-id>/request.md`
  using the workflow's request template.
- Request IDs follow the pattern `R-NNN` (sequential, zero-padded).
- When evaluating, always load the full rubric from the workflow
  definition. Do not evaluate from memory.
- After evaluation, surface the recommendation clearly but do not
  make the accept/reject decision — that belongs to the decision-maker.
```

Skills

Reusable AI Capabilities You Can Build and Share

What a Skill Is

- A packaged prompt — a structured Markdown file defining a specific, reusable capability
- Tells the agent *exactly* how to approach a task: steps, constraints, what good output looks like
- Skills are composable — build a library, invoke only what's relevant
- Named procedures your agent can call, not instructions you re-explain every session

How Skills Get Invoked

A skill's `description` frontmatter is what the agent reads to decide whether to load it.

```
---
name: test-driven-development
description: Use when implementing any feature or bugfix,
             before writing implementation code
---

## Steps
1. Write a failing test first …
```

The agent sees the `description` only – the full skill body stays out of context until invoked.

You control selectivity through the description:

- Broad trigger → loads often, keeps agent behavior consistent across many tasks
- Narrow trigger → loads rarely, reserves specialized guidance for specific moments
- No trigger → agent guesses – avoid this

Skills are distributed through GitHub and discoverable at **skills.sh**

Installing Skills

```
# No global install required
node --version   # 18+

# Install a single skill
npx skills add https://github.com/hashicorp/agent-skills \
  --skill terraform-style-guide

# Install all skills from a repo
npx skills add https://github.com/hashicorp/agent-skills

# See what's installed
npx skills list
```

Skills are stored in `.claude/skills/` and committed to the repo — the whole team shares them.

What Makes a Great Skill

- **Single responsibility** — one skill, one domain
- **Opinionated, not optional** — makes decisions: "use `for_each` over `count`"
- **Concrete over abstract** — show examples, name libraries, give exact patterns
- **Tells the agent what NOT to do** — constraints are as valuable as instructions
- **Written for the agent, not a human** — imperative, direct, no hedging
- **Tested** — invoke it, see what the agent does, refine until reliable

Demo: Skills in Action

```
# See TeamFabric's installed skills
ls TeamFabric/Fabric/.claude/skills/

# Read the ingestion skill
cat TeamFabric/Fabric/.claude/skills/ingestion.md
```

> Drop a raw meeting note into `TeamFabric/Example/staging/` and run:
>
> `/ingest staging`

TeamFabric: A Skill File

Clear purpose. Explicit trigger conditions. Step-by-step procedure. No ambiguity.

```
# Skill: Content Ingestion

## Purpose
Process incoming content into Fabric's structured entity model.
The core "people dump raw content in, AI organizes it" capability.

## Three Ingestion Paths

### Quick File
User provides content plus an entity hint.
Trigger: "this is for R-42" or "file this under WI-1234"
Procedure:
1. Load the referenced entity's header and recent context log.
2. Summarize the content against that entity's context.
3. Present the proposed context log entry for confirmation.
4. On confirmation, append to the entity's context log.
5. Set the entity's staleness flag if summary may affect first-class fields.
```

TeamFabric: Skills vs. Rules

A rule constrains. A skill enables.

	Rule (`fabric-triage.md`)	Skill (`ingestion.md`)
Loaded	When triage module enabled	When ingestion task detected
Says	"Always load rubric from file"	"Here are the three ingestion paths"
Role	Constrains agent behavior	Gives agent a procedure to execute
Scope	Applies to all triage work	Invoked for this specific capability

Commands

Automating Workflows, Not Just Guiding Them

What a Command Is

- A Markdown file in `.claude/commands/` that becomes a `/slash-command`
- When invoked, its contents are injected directly into the conversation as a prompt
- Unlike skills, commands can include **executable bash blocks** that run before Claude processes the request
- Think of them as scripts with an AI attached: first automate, then reason

Skills vs. Commands

	Skills	Commands
Purpose	Teach the agent a capability	Automate a repeatable workflow
Trigger	Agent detects relevance	User invokes with `/`
Execution	Pure prompt	Can run bash before AI engages
Best for	Coding standards, patterns	Ops tasks, doc generation, scaffolding

Skills make the agent smarter about *how* to do something.
Commands make the agent faster at *doing* something specific, every time.

Demo: Commands in Action

```
# Read the status command
cat TeamFabric/Fabric/.claude/commands/status.md

# Run status against the example instance
# (open Example/ in Claude Code, then)
/status

# Generate a mindmap report
/report mindmap
```

TeamFabric: A Command File

Instructions written directly to the AI. The agent reads this file and follows it exactly.

```
# /status - Team Status Summary

## Purpose
Quick factual snapshot of current team state.
Numbers and lists, not narrative.

## Behavior

1. Load:
   - team/team.md (members, allocation, current state)
   - Active member profiles (capacity adjustments for current/upcoming periods)
   - Request counts if available

2. Output a concise summary:

Team: [name]
Effective Capacity: [X] FTE

Active Members:
  [name] - [role] - [allocation]%

## Notes
```

Managing Context

Context Is Not Free

Every token in the context window costs money — and attention.

Cost

- Most providers charge per input token
- A full session with large files costs real money
- Long contexts make the agent slower to respond
- Accumulated noise degrades output quality

Attention

- Models weight recent tokens more than old ones
- A stale error from 20 messages ago still influences output
- Mid-task corrections get buried under conversation history
- A dirty context is a distracted agent

> The cleanest agent is one that starts fresh with exactly the context it needs.

Controlling Your Context

Clear it
`/clear`

Wipe the conversation. Your guardrails (constitution, rules, skills) reload automatically — you don't lose your standards, only the noise.

Best for: starting a new task in the same session.

Compact it
`/compact`

Summarize the conversation into a compressed context block. Keeps continuity, reduces token count.

Best for: long-running tasks where you need to preserve state but are burning through tokens.

Shelf it
`/rename` then `claude --continue`

Give the session a meaningful name. Later, `claude --continue [name]` resumes it exactly where you left off.

Best for: "I'll come back to this" — a branch you're not done with yet.

Quick escape: `Ctrl+Z` suspends Claude, drops you to a plain shell. Run whatever you need. `fg` brings Claude back with the session intact.

The Context Hygiene Habit

```
Finish a task → /clear → Start next task
```

What survives a `/clear`

- Your constitution (CLAUDE.md)
- Your rules (`.claude/rules/`)
- Your skills (`.claude/skills/`)
- Your memory (`.claude/memory/`)

What gets wiped

- Conversation history
- Tool output accumulation
- Mid-session corrections
- The model's "current mood"

> Your guardrails are durable. The conversation is disposable.

Bringing It All Together

When to Reach for Each Tool

Tool	Answers	Signal you need it
Spec	What to build right now	Agent keeps building the wrong thing
Constitution	How we always work	Re-explaining setup every session
Memory	What has this agent learned	Correcting the same mistake repeatedly
Rule	What applies in this context	CLAUDE.md only matters for some files
Skill	How to do this type of task	Writing the same multi-paragraph prompt
Command	Run this specific workflow	Shell script + explaining output every time

These tools compound. Together they shift the team from *prompt engineering* to *engineering the agent*.

Q&A

Eric Winter
Research Genomics